



福建師範大學
FUJIAN NORMAL UNIVERSITY

《机器学习》

【第九章-人工神经网络】

深度学习与多层感知机
黄锐泓

本节要点

1. 深度学习与神经网络概述
2. 神经元模型
3. 激活函数
4. 多层感知机



深度学习与神经网络概述

神经网络 (Neural Network)

人工神经网络 (Artificial Neural Networks, 简称为ANNs) 也简称为神经网络 (NNs), 它从信息处理角度对人脑神经元网络进行抽象, 建立某种简单模型, 按不同的连接方式组成不同的网络。

人工神经网络借鉴了生物神经网络的思想, 是超级简化版的生物神经网络。以工程技术手段模拟人脑神经系统的结构和功能, 通过大量的非线性并行处理器模拟人脑中众多的神经元, 用处理器复杂的连接关系模拟人脑中众多神经元之间的突触行为。



深度学习与神经网络概述

神经网络 (Neural Network)

人工神经网络 (Artificial Neural Networks, 简称为ANNs) 也简称为神经网络 (NNs), 它从信息处理角度对人脑神经元网络进行抽象, 建立某种简单模型, 按不同的连接方式组成不同的神经网络。

人工神经网络借鉴了生物神经系统的结构, 是超级简化版的生物神经网络。

以工程技术手段模拟生物神经系统的结构和功能, 通过大量的非线性并行处理器模拟人脑中众多的神经元, 用处理器复杂的连接关系模拟人脑中众多神经元之间的突触行为。

是一个数学模型



深度学习与神经网络概述

深度学习（Deep Learning）

1. 无标准定义，但总的来说，深度学习是机器学习（ML）的一种，主要可以看作是人工神经网络（ANNs）的高级模型。这些技术被用作实现人工智能（AI）的工具。
2. 深度学习是机器学习的分支，是一种以人工神经网络为架构，对数据进行特征学习的算法



深度学习与神经网络概述

深度学习（Deep Learning）

1. 无标准定义，但总的来说，深度学习是机器学习（ML）的一种，主要可以看作是人工神经网络（ANNs）的高级模型。深度学习技术被用作实现人工智能（AI）的工具。

2. 深度学习是机器学习的一种，它以人工神经网络为架构，对数据进行特征学习的算法。

是高级、复杂的神经网络

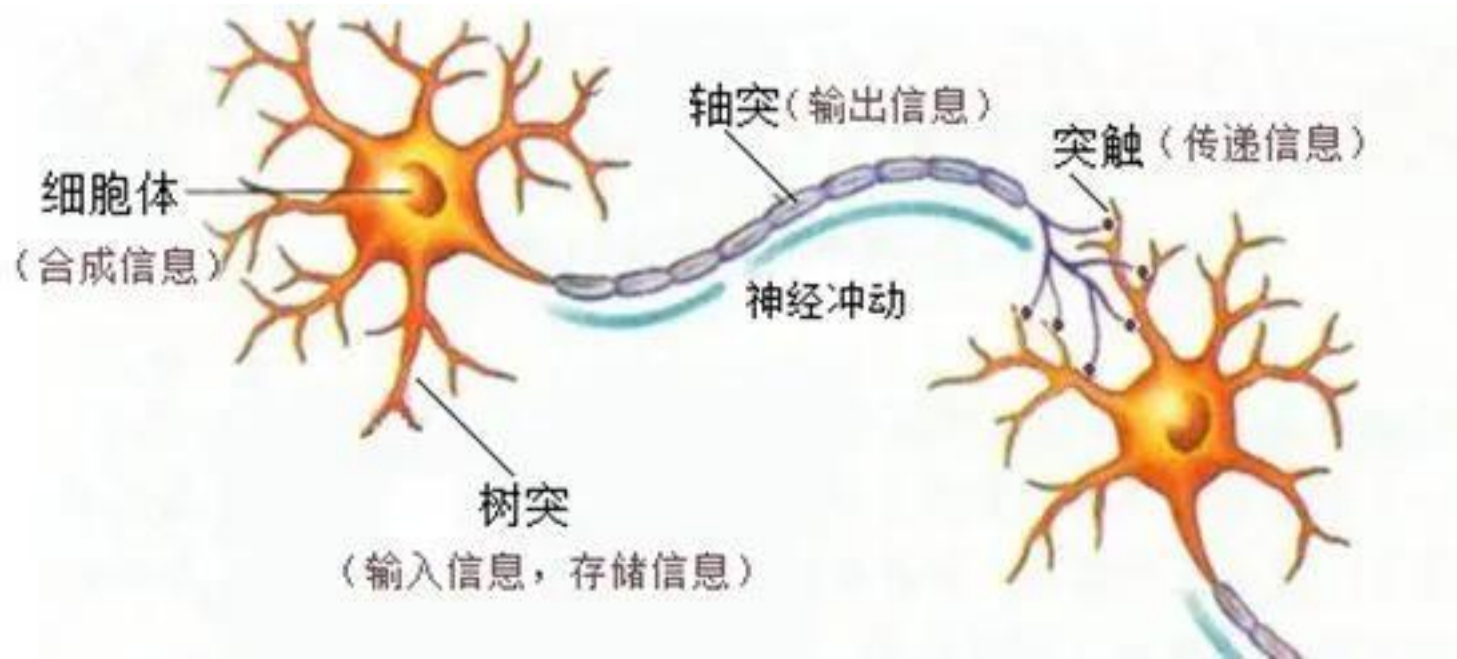


神经元模型

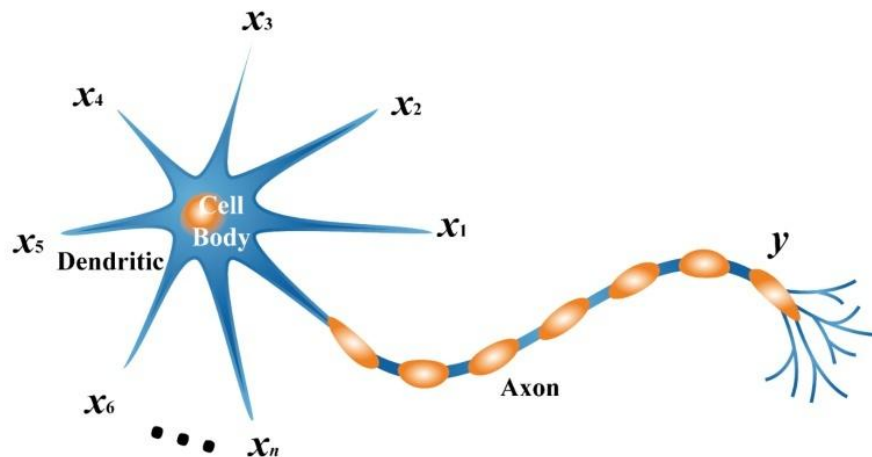
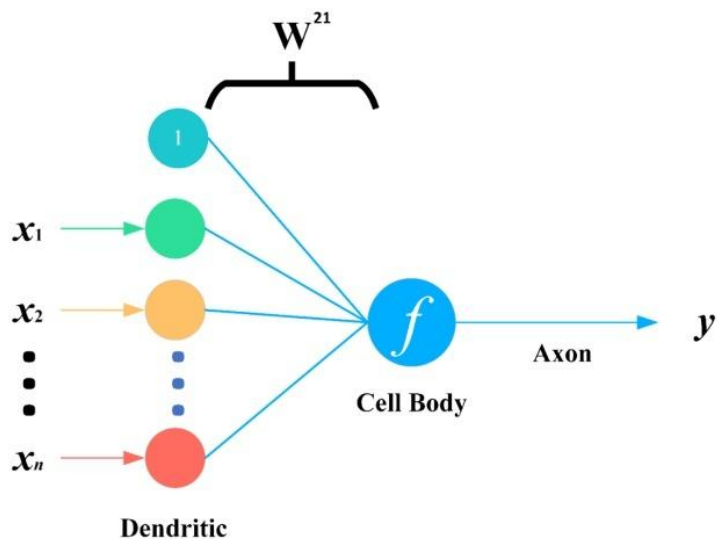
神经网络的基本组成单位是神经元模型，用于模拟生物神经网络中的神经元。生物神经网络中，神经元的功能是感受刺激传递兴奋。每个神经元通过树突接受来自其他被激活神经元的，通过轴突释放出来的化学递质，改变当前神经元内的电位，然后将其汇总。当神经元内的电位累计到一个水平时就会被激活，产生动作电位，然后通过轴突释放化学物质。



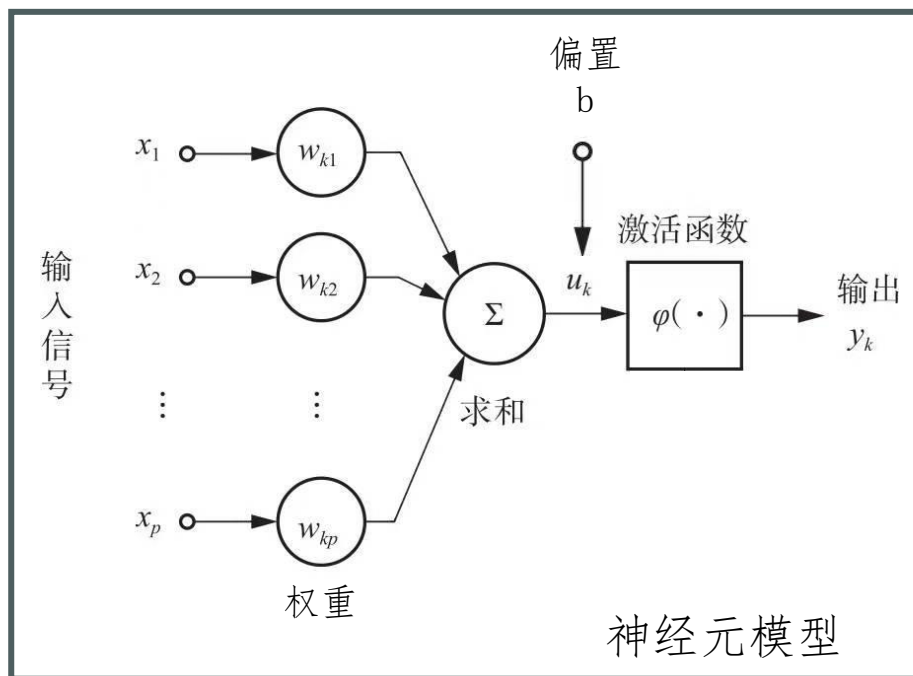
神经元模型



神经元模型



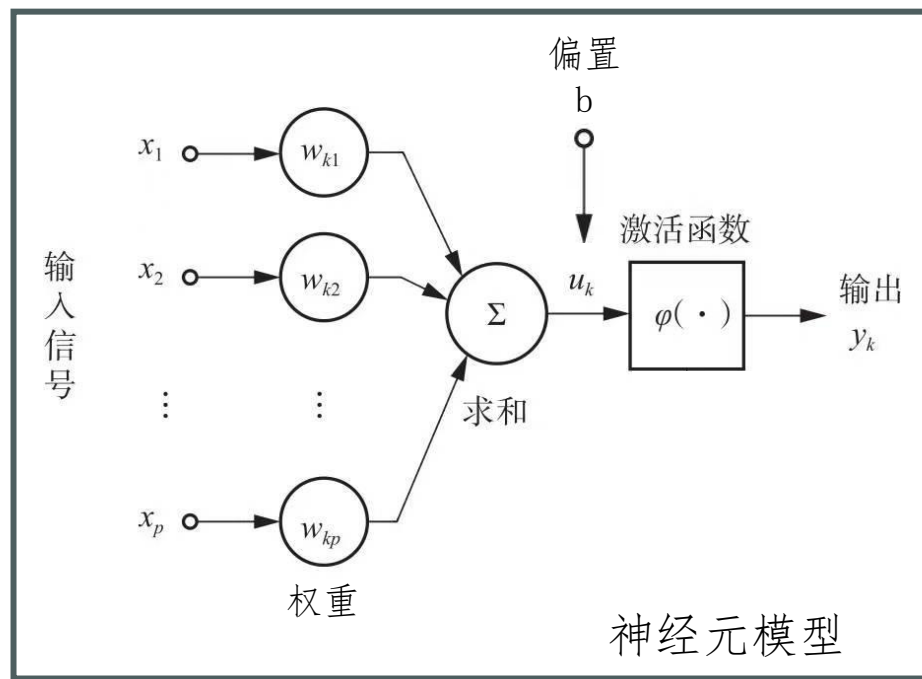
神经元模型



神经元模型

神经元模型的基本组成：

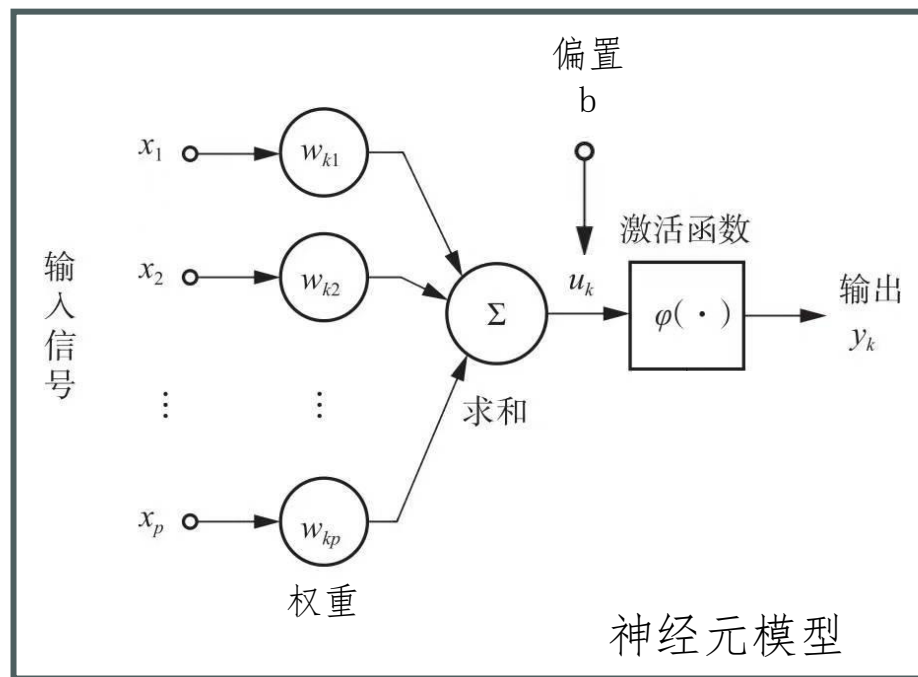
1. 突触或链接： 上面的 x_p 是神经元的输入，相当于树突接收的多个外部刺激。 w_{kp} 是每个输入对应的权重，代表了每个特征的重要程度，它对应于每个输入特征，影响着每个输入 x_p 的刺激强度。



神经元模型

神经元模型的基本组成：

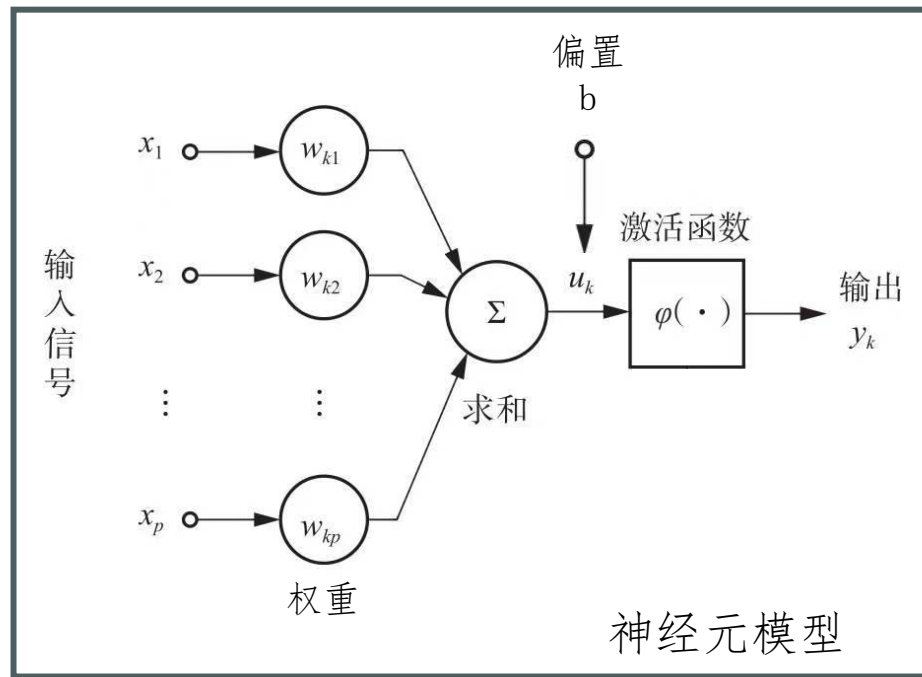
2. 求和单元 Σ ：用于求取各输入信号 x_p 的加权 w_{kp} 和，也就是各输入信号的线性组合。



神经元模型

神经元模型的基本组成：

3. 非线性的激活函数：起非线性映射作用，并将神经元输出幅度限制在一定的范围内，一般会在 $(0, 1)$ 或 $(-1, 1)$ 之间。



神经元模型

右图的数学表达：

$$u_k = \sum_{p=1}^m w_{kp} x_p \quad (1)$$

$$y_k = \varphi(u_k + b) \quad (2)$$

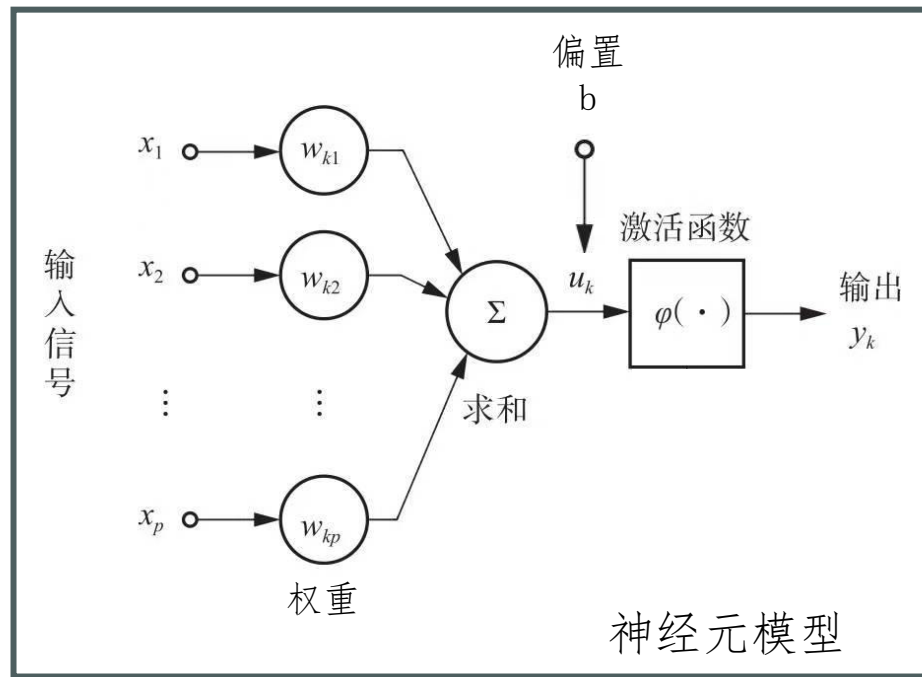
其中，

$x_1, x_2, \dots, x_p$ 为输入信号；

$w_{k1}, w_{k2}, \dots, w_{kp}$ 为神经元k对各信号的权重；

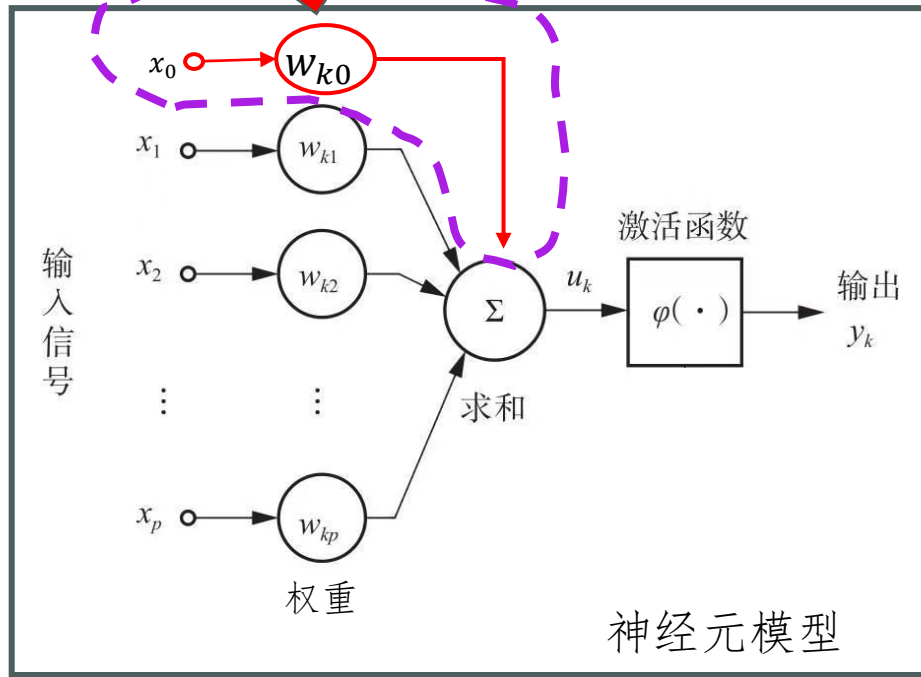
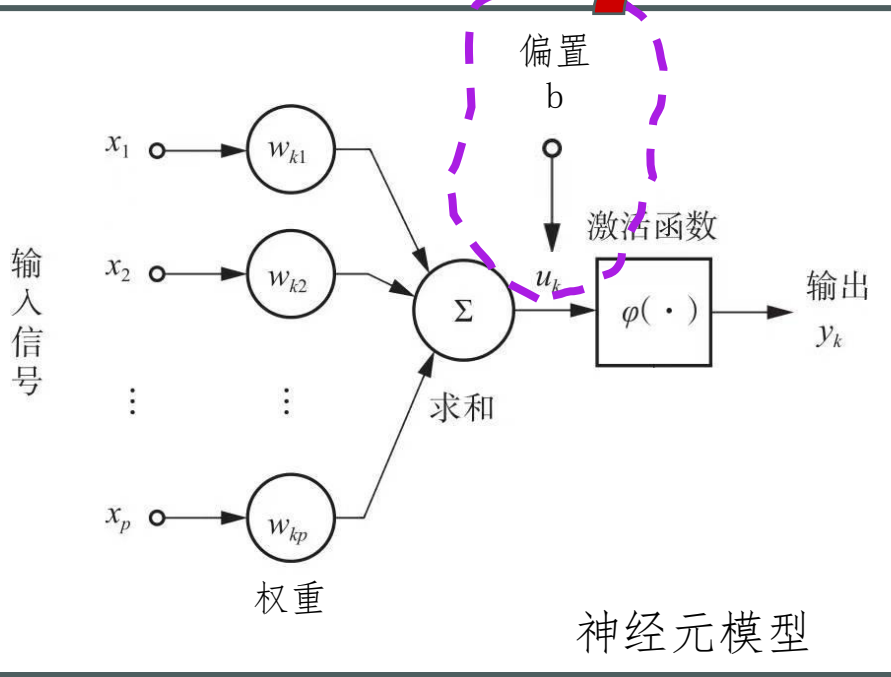
u_k 是输入信号的线性组合， b 是偏置；

$\varphi(\cdot)$ 是 **激活函数**



神经元模型

对神经元模型进行化简：



令 $x_0 = 1, w_{k0} = b$, 则：

$$u_k = \sum_{p=1}^m w_{kp} x_p \quad (1)$$

$$y_k = \varphi(u_k + b) \quad (2)$$



$$u_k = \sum_{p=0}^m w_{kp} x_p \quad (1)$$

$$y_k = \varphi(u_k) \quad (2)$$



常见激活函数

1. 什么是激活函数 (Activation Function) ?

又称非线性映射函数或是隐藏单元，是神经网络中最主要的组成部分之一。“激活”这个名字来自于生物上的神经网络结构。人工神经网络最初是受到生物上的启发而设计出了神经元这种单位结构，在每个神经元中，需要一个“开关”来决定该神经元的信息是否会被传递到其相连的神经元去，这个“开关”在这里也就是激活函数。



常见激活函数

1. 什么是激活函数 (Activation Function) ?

2. 激活函数的作用是什么?

对输入的信息进行非线性变化

回顾神经元模型的数学表达:

$$u_k = \sum_{p=0}^m w_{kp} x_p$$

线性累加过程

$$y_k = \varphi(u_k)$$

通过激活函数 φ 进行非线性变化换

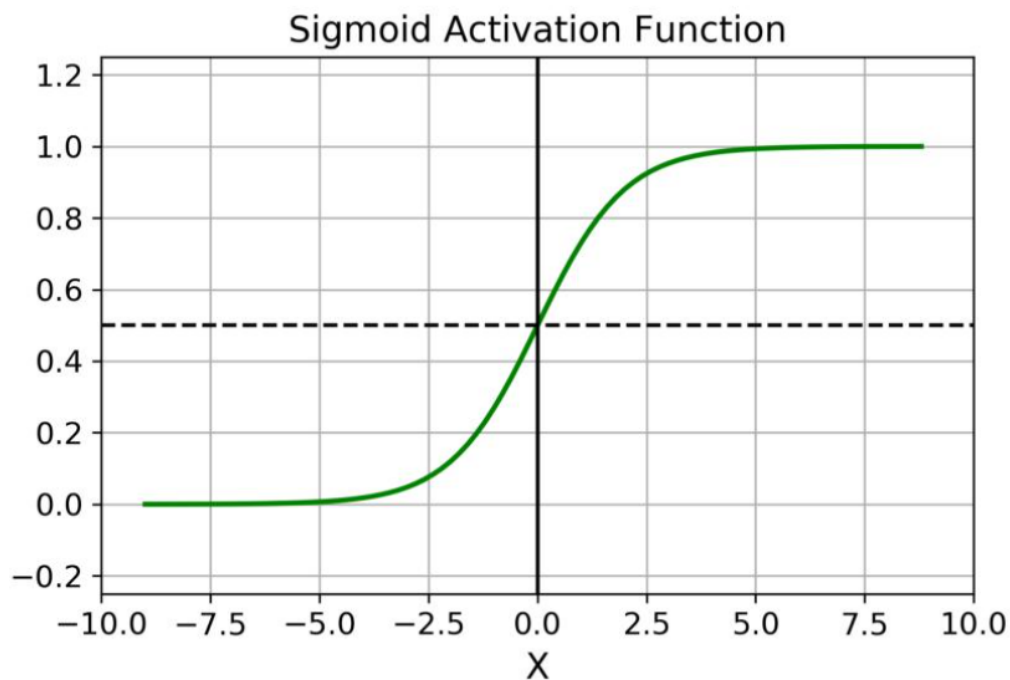


常见激活函数

Sigmoid

Sigmoid函数也叫Logistic函数，用于隐藏层的输出，输出在(0,1)之间，它可以将一个实数映射到(0,1)的范围内，可以用来做二分类。

$$\text{Sigmoid}(x) = \frac{1}{1 + e^{-x}}$$



常见激活函数

Sigmoid的优缺点

优点：

1. 将很大范围内的输入特征值压缩到 $0 \sim 1$ 之间，使得在深层网络中可以保持数据幅度不会出现较大的变化，而Relu函数则不会对数据的幅度作出约束；
2. 在物理意义上最为接近生物神经元；
3. 根据其输出范围，该函数适用于将预测概率作为输出的模型。



常见激活函数

Sigmoid的优缺点

缺点：

1. 当输入非常大或非常小的时候（**饱和**），输出基本为常数，即变化非常小，进而导致梯度接近于0（**梯度消失**）；
2. 输出不是0均值，进而导致后一层神经元将得到上一层输出的非0均值的信号作为输入。随着网络的加深，会改变原始数据的分布趋势；
3. 幂运算相对耗时。

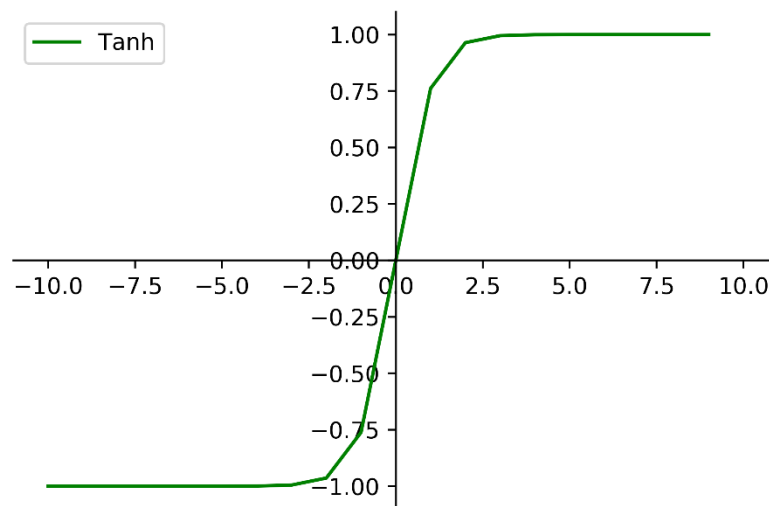


常见激活函数

Tanh

Tanh 激活函数又叫作双曲正切激活函数（hyperbolic tangent activation function）。

$$\text{Tanh}(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



常见激活函数

Tanh的优缺点

优点：

1. 解决了上述的Sigmoid函数输出不是0均值的问题；
2. Tanh函数的导数取值范围在 $0 \sim 1$ 之间，优于sigmoid函数的0.25，一定程度上缓解了梯度消失的问题；
3. Tanh函数在原点附近与 $y=x$ 函数形式相近，当输入的激活值较低时，可以直接进行矩阵运算，训练相对容易；



常见激活函数

Tanh的优缺点

缺点：

与sigmoid类似，梯度消失（gradient vanishing）的问题和幂运算的问题仍然存在。

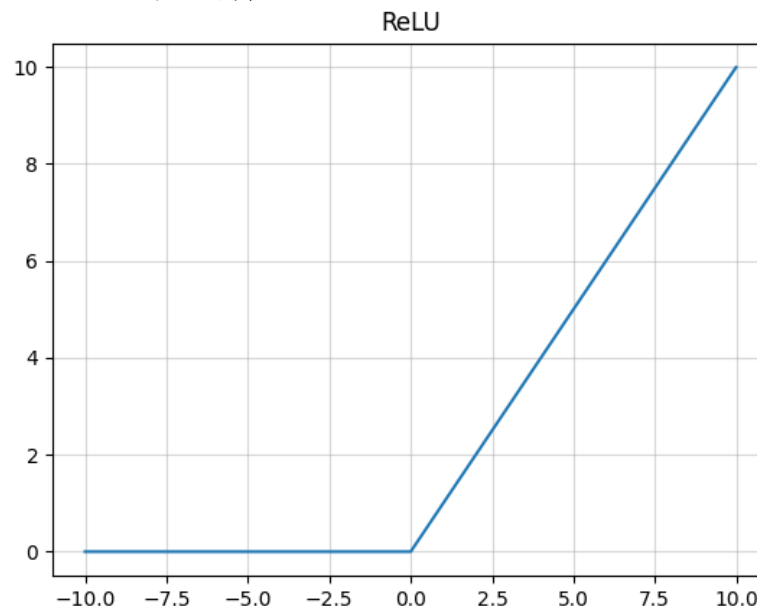


常见激活函数

ReLU

ReLU全称是Rectified Linear Unit，修正线性单元。ReLU是Krizhevsky、Hinton等人在2012年《ImageNet Classification with Deep Convolutional Neural Networks》论文中提出的一种线性且不饱和的激活函数。

$$\text{ReLU}(x) = \max(0, x)$$



常见激活函数

ReLU的优缺点

优点：

1. 相较于sigmoid函数来看，在输入为正时，Relu函数不存在饱和问题，即解决了**梯度消失**的问题，使得深层网络可训练；
2. 计算速度非常快，只需要判断输入是否大于0值；
3. 收敛速度远快于sigmoid以及Tanh函数；
4. Relu输出会使一部分神经元为0值，在带来网络稀疏性的同时，也减少了参数之间的关联性，一定程度上缓解了过拟合的问题；



常见激活函数

ReLU的优缺点

缺点：

1. Relu函数的输出也不是以0为均值的函数；
2. 存在Dead ReLU Problem，即某些神经元($x < 0$)可能永远不会被激活，进而导致相应参数一直得不到更新；
3. 当输入为正值，导数为1，在“链式反应”中，不会出现梯度消失，但梯度下降的强度则完全取决于权值的乘积，如此可能会导致**梯度爆炸**问题；

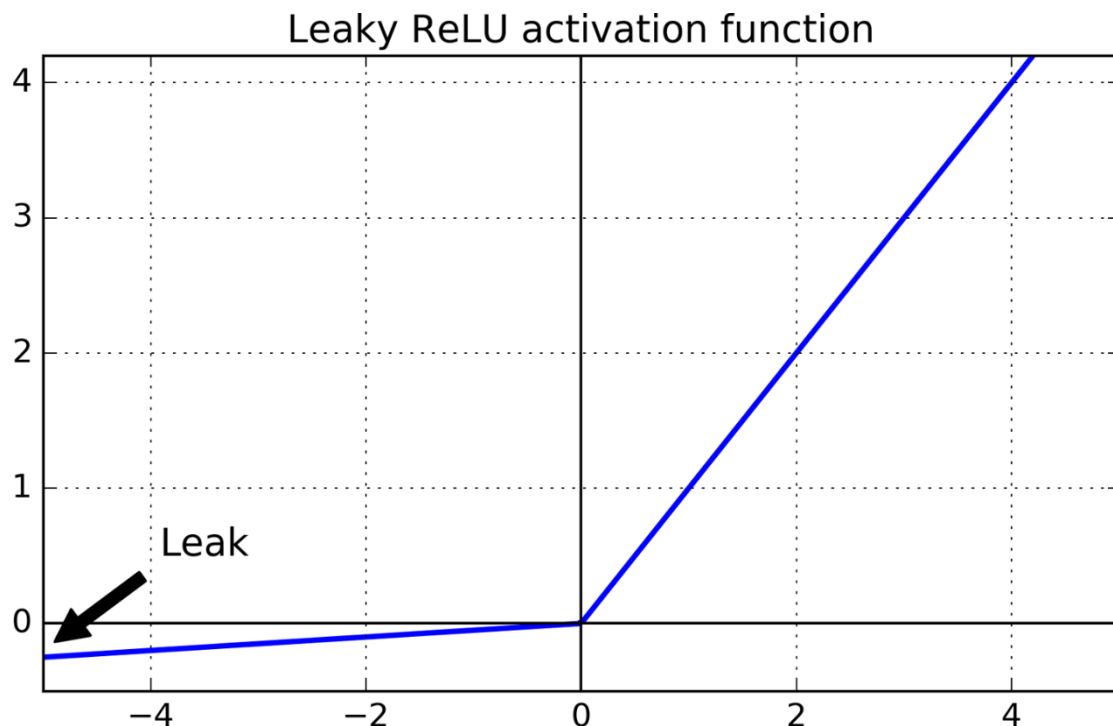


常见激活函数

为了解决Dead ReLU Problem —— Leaky ReLU

$$\text{Leaky_ReLU}(x) = \max(\alpha x, x)$$

α 值一般设置为一个较小值，如0.01

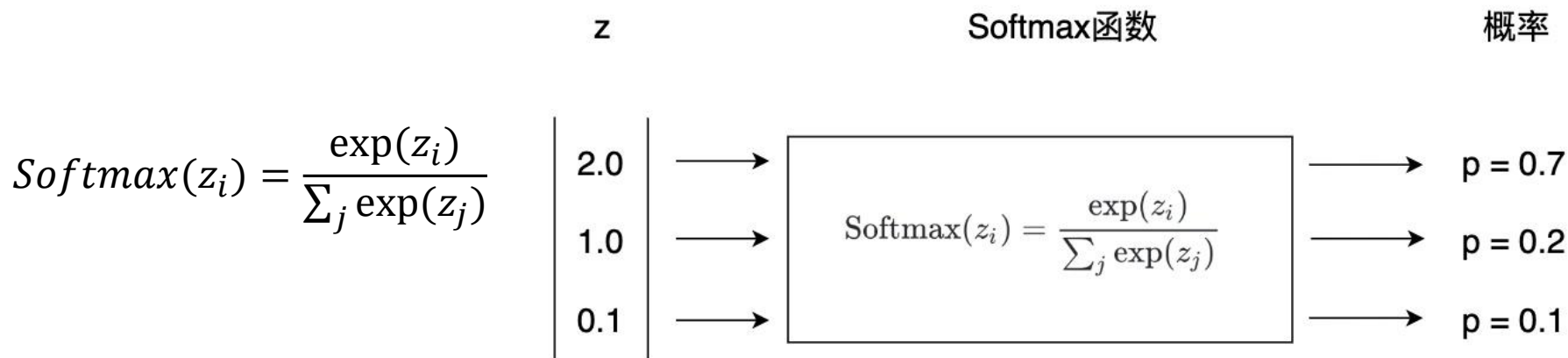


常见激活函数

Softmax

Softmax可以将一个数值向量归一化为一个概率分布向量，且各个概率之和为1。

Softmax可以用来作为神经网络的最后一层，用于多分类问题的输出。Softmax层常常和交叉熵损失函数一起结合使用。



多层感知机 (MLP)

多层感知机 (Multi-Layer Perceptron, MLP) 通过堆叠多个**神经元模型**组成，是最简单的神经网络模型。

多层感知机是深度神经网络 (Deep Neural Networks, DNN) 的基础算法



多层感知机 (MLP)

多层感知机 (Multi-Layer Perceptron, MLP) 通过堆叠多个**神经元模型**组成，是最简单的神经网络模型。

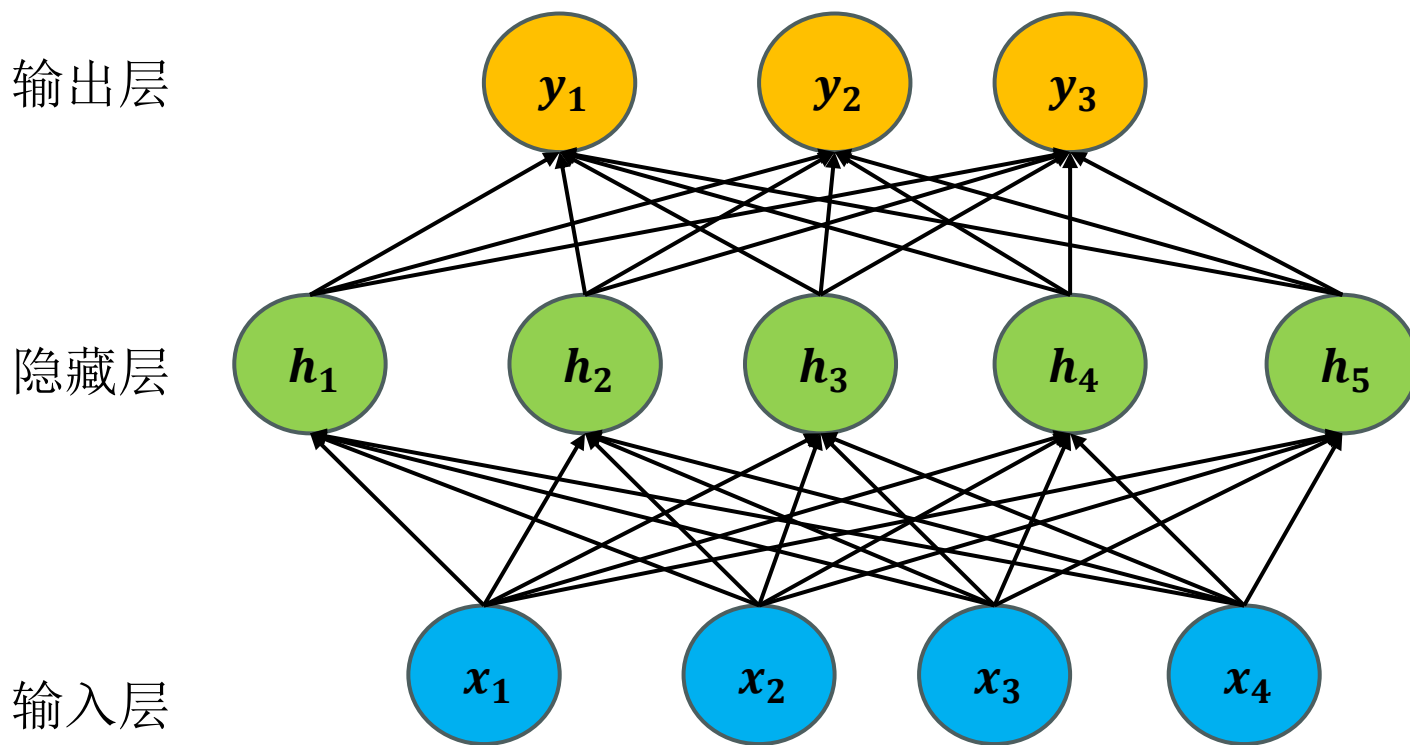
多层感知机是深度神经网络 (Deep Neural Networks, DNN) 的基础算法。

多层感知机包含输入层、**至少一个隐藏层**与输出层。因此，多层感知机至少为三层。



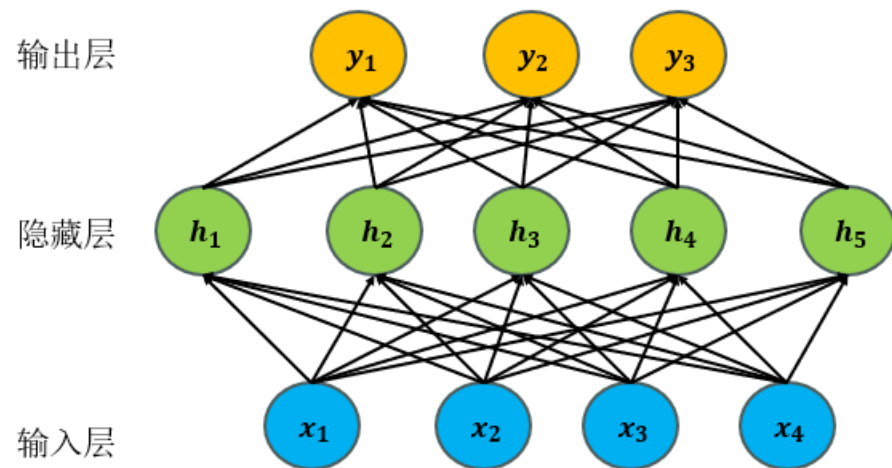
多层感知机 (MLP)

以三层的MLP为例：



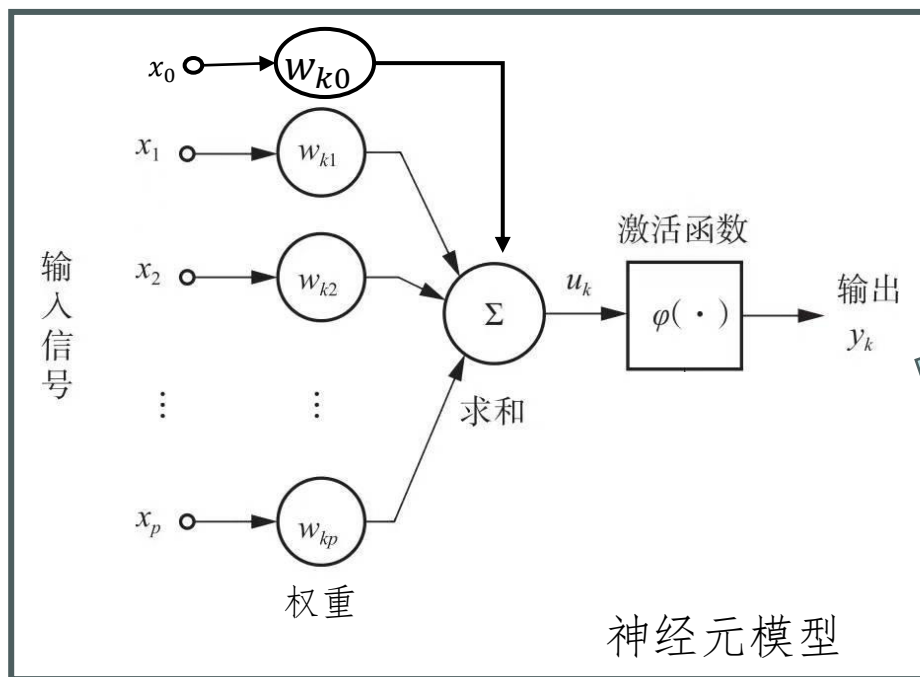
多层感知机 (MLP)

图中 $\mathbf{X} = (x_1, x_2, x_3, x_4)$ 称为输入层，包含四个节点，对应数据的四个特征； $\mathbf{Y} = (y_1, y_2, y_3)$ 称为输出层，包含三个节点；中间为隐藏层 $\mathbf{H} = (h_1, h_2, h_3, h_4, h_5)$ ，神经元节点个数为 5。输入层与隐藏层、隐藏层与输出层之间的神经元节点两两都有连接（常称之为 **全连接层**）。

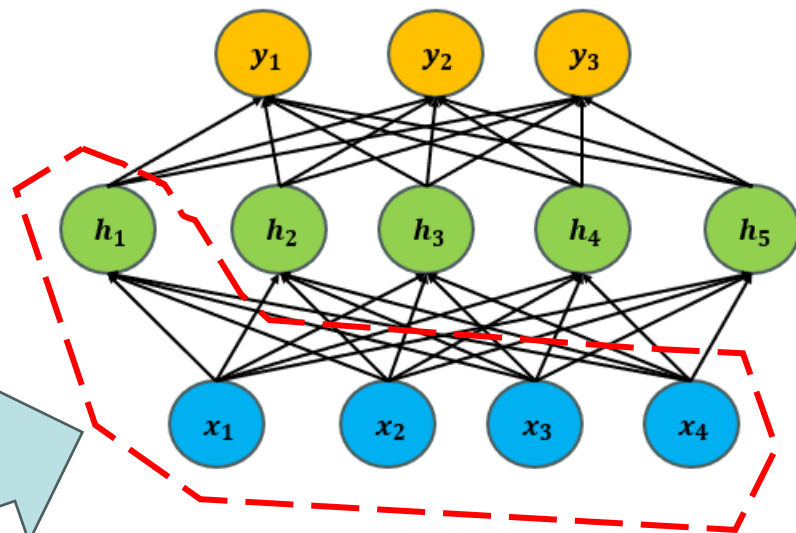


多层感知机 (MLP)

对比MLP与神经元模型



近似



类似的上述MLP中 h_k 的输出的数学表达式为：

$$u_k = \sum_{p=0}^m w_{kp} x_p \quad (1)$$

$$h_k = \varphi(\sum_{p=0}^4 w_{kp} x_p)$$

$$y_k = \varphi(u_k) \quad (2)$$



多层感知机 (MLP)

MLP的数学表达形式

假设隐藏层 $\mathbf{H}$ 的激活函数为Sigmoid,

输出层 $\mathbf{Y}$ 的激活函数为Softmax函数。

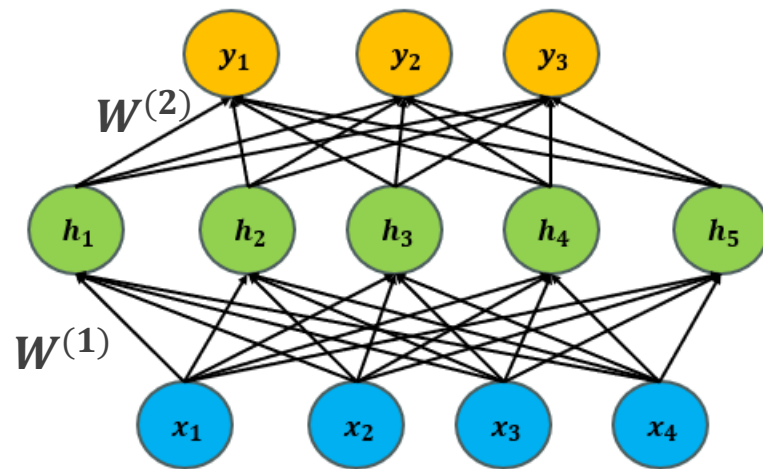
记输入层到隐藏层的参数为 $\mathbf{W}^{(1)}$; 记隐

藏层到输出层的参数为 $\mathbf{W}^{(2)}$, 则整个

多层感知机神经网络可以描述为:

$$\mathbf{H} = \text{sigmoid}(\mathbf{W}^{(1)\top} \mathbf{X})$$

$$\mathbf{Y} = \text{Softmax}(\mathbf{W}^{(2)\top} \mathbf{H})$$



其中 h_k 的输出的数学表达式为:

$$h_k = \text{sigmoid}(\sum_{p=0}^4 w^{(1)}_{kp} x_p)$$

y_k 的输出的数学表达式为:

$$y_k = \text{softmax}(\sum_{j=0}^5 w^{(2)}_{kj} h_j)$$



小结

1. 深度学习与神经网络概述
2. 神经元模型（神经网络的基础）
3. 激活函数（神经网络的关键）
4. 多层感知机(结构特点)



谢谢！

